TypeResolver and FqsenResolver

The specification on types in DocBlocks (PSR-5) describes various keywords and special constructs but also how to statically resolve the partial name of a Class into a Fully Qualified Class Name (FQCN).

PSR-5 also introduces an additional way to describe deeper elements than Classes, Interfaces and Traits called the Fully Qualified Structural Element Name (FQSEN). Using this it is possible to refer to methods, properties and class constants but also functions and global constants.

This package provides two Resolvers that are capable of

1. Returning a series of Value Object for given expression while resolving any partial class names, and
2. Returning an FQSEN object after resolving any partial Structural Element Names into Fully Qualified Structural Element names.

Installing

The easiest way to install this library is with Composer using the following command:

```
$ composer require phpdocumentor/type-resolver
```

Examples

Ready to dive in and don't want to read through all that text below? Just consult the examples folder and check which type of action that your want to accomplish.

On Types and Element Names

This component can be used in one of two ways

1. To resolve a Type or
2. To resolve a Fully Qualified Structural Element Name

The big difference between these two is in the number of things it can resolve.

The TypeResolver can resolve:

- a php primitive or pseudo-primitive such as a string or void (`@var string` or `@return void`).
- a composite such as an array of string (`@var string[]`).

- a compound such as a string or integer (`@var string|integer`).
- an array expression (`@var (string|TypeResolver)[]`)
- an object or interface such as the TypeResolver class (`@var TypeResolver` or `@var \phpDocumentor\Reflection\TypeResolver`)

  please note that if you want to pass partial class names that additional steps are necessary, see the chapter `Resolving partial classes and FQSENs` for more information.

Where the FqsenResolver can resolve:

- Constant expressions (i.e. `@see \MyNamespace\MY_CONSTANT`)
- Function expressions (i.e. `@see \MyNamespace\myFunction()`)
- Class expressions (i.e. `@see \MyNamespace\MyClass`)
- Interface expressions (i.e. `@see \MyNamespace\MyInterface`)
- Trait expressions (i.e. `@see \MyNamespace\MyTrait`)
- Class constant expressions (i.e. `@see \MyNamespace\MyClass::MY_CONSTANT`)
- Property expressions (i.e. `@see \MyNamespace\MyClass::$myProperty`)
- Method expressions (i.e. `@see \MyNamespace\MyClass::myMethod()`)

Resolving a type

In order to resolve a type you will have to instantiate the class `\phpDocumentor\Reflection\TypeResolver` and call its `resolve` method like this:

```
$typeResolver = new \phpDocumentor\Reflection\TypeResolver();
$type = $typeResolver->resolve('string|integer');
```

In this example you will receive a Value Object of class `\phpDocumentor\Reflection\Types\Compound` that has two elements, one of type `\phpDocumentor\Reflection\Types\String_` and one of type `\phpDocumentor\Reflection\Types\Integer`.

The real power of this resolver is in its capability to expand partial class names into fully qualified class names; but in order to do that we need an additional `\phpDocumentor\Reflection\Types\Context` class that will inform the resolver in which namespace the given expression occurs and which namespace aliases (or imports) apply.

Resolving nullable types

Php 7.1 introduced nullable types e.g. `?string`. Type resolver will resolve the original type without the nullable notation `?` just like it would do without the `?`. After that the type is wrapped in a `\phpDocumentor\Reflection\Types\Nullable` object. The `Nullable` type has a method to fetch the actual type.

Resolving an FQSEN

A Fully Qualified Structural Element Name is a reference to another element in your code bases and can be resolved using the `\phpDocumentor\Reflection\FqsenResolver` class' `resolve` method, like this:

```
$fqsenResolver = new \phpDocumentor\Reflection\FqsenResolver();
$fqsen = $fqsenResolver->resolve('\phpDocumentor\Reflection\FqsenResolver::resolve()');
```

In this example we resolve a Fully Qualified Structural Element Name (meaning that it includes the full namespace, class name and element name) and receive a Value Object of type `\phpDocumentor\Reflection\Fqsen`.

The real power of this resolver is in its capability to expand partial element names into Fully Qualified Structural Element Names; but in order to do that we need an additional `\phpDocumentor\Reflection\Types\Context` class that will inform the resolver in which namespace the given expression occurs and which namespace aliases (or imports) apply.

Resolving partial Classes and Structural Element Names

Perhaps the best feature of this library is that it knows how to resolve partial class names into fully qualified class names.

For example, you have this file:

```
namespace My\Example;

use phpDocumentor\Reflection\Types;

class Classy
{
    /**
     * @var Types\Context
     * @see Classy::otherFunction()
     */
    public function __construct($context) {}

    public function otherFunction(){}
}
```

Suppose that you would want to resolve (and expand) the type in the `@var` tag and the element name in the `@see` tag.

For the resolvers to know how to expand partial names you have to provide a bit of *Context* for them by instantiating a new class named `\phpDocumentor\Reflection\Types\Context` with the name of the namespace and the aliases that are in play.

Creating a Context

You can do this by manually creating a Context like this:

```
$context = new \phpDocumentor\Reflection\Types\Context(
    '\My\Example',
    [ 'Types' => '\phpDocumentor\Reflection\Types']
);
```

Or by using the `\phpDocumentor\Reflection\Types\ContextFactory` to instantiate a new context based on a Reflector object or by providing the namespace that you'd like to extract and the source code of the file in which the given type expression occurs.

```
$contextFactory = new \phpDocumentor\Reflection\Types\ContextFactory();
$context = $contextFactory->createFromReflector(new ReflectionMethod('\My\Example\Classy', '
```

or

```
$contextFactory = new \phpDocumentor\Reflection\Types\ContextFactory();
$context = $contextFactory->createForNamespace('\My\Example', file_get_contents('My/Example/
```

Using the Context

After you have obtained a Context it is just a matter of passing it along with the `resolve` method of either Resolver class as second argument and the Resolvers will take this into account when resolving partial names.

To obtain the resolved class name for the `@var` tag in the example above you can do:

```
$typeResolver = new \phpDocumentor\Reflection\TypeResolver();
$type = $typeResolver->resolve('Types\Context', $context);
```

When you do this you will receive an object of class `\phpDocumentor\Reflection\Types\Object_` for which you can call the `getFqsen` method to receive a Value Object that represents the complete FQSEN. So that would be `phpDocumentor\Reflection\Types\Context`.

> Why is the FQSEN wrapped in another object `Object_`?
>
> The resolve method of the TypeResolver only returns object with the interface `Type` and the FQSEN is a common type that does not represent a Type. Also: in some cases a type can represent an "Untyped Object", meaning that it is an object (signified by the `object` keyword) but does not refer to a specific element using an FQSEN.

Another example is on how to resolve the FQSEN of a method as can be seen with the `@see` tag in the example above. To resolve that you can do the following:

```
$fqsenResolver = new \phpDocumentor\Reflection\FqsenResolver();
$type = $fqsenResolver->resolve('Classy::otherFunction()', $context);
```

Because Classy is a Class in the current namespace its FQSEN will have the `My\Example` namespace and by calling the `resolve` method of the FQSEN Resolver you will receive an `Fqsen` object that refers to `\My\Example\Classy::otherFunction()`.